ELHAGE ET AL.: TOY MODELS OF SUPERPOSITION, PART II

MINZHE LI

OCTOBER 31, 2026

1 Recap

Notation from Part I

- Features $x \in \mathbf{R}^n$: $x_i = 0$ with probability S (the sparsity), otherwise x_i is uniform on $[0, 1]$. Feature i has importance I_i.
- ReLU output model: $h = Wx \in \mathbf{R}^m$ and $x' = \text{ReLU}(W^\top Wx + b)$, where $W \in \mathbf{R}^{m \times n}$ has the feature directions W_i as columns. Loss: $L = \sum_x \sum_i I_i(x_i - x'_i)^2$.
- Superposition means representing more features than there are dimensions. Feature i is represented if $\|W_i\|^2 \approx 1$. Feature dimensionality: $D_i = \|W_i\|^2 / \sum_j (\hat{W}_i \cdot W_j)^2$. Dimensions per feature: $D^* = m/\|W\|_F^2$.
- Results: superposition appears as features get sparser, through phase changes. Features form polytopes: antipodal pairs ($D_i = 1/2$), triangles (2/3), pentagons (2/5), tetrahedra (3/4). Correlated features prefer orthogonal directions, anticorrelated ones antipodal.

2 Superposition and Learning Dynamics

The trained models end in simple, discrete structures, so one can ask how training reaches them (p. 26). The paper reports two phenomena and leaves a fuller study for later work.

Energy-level jumps (pp. 26–27). In the uniform setting of Part I (many features of equal importance and sparsity), the authors pick a model whose features all end in antipodal pairs and follow every D_i during training (Figure 1). The D_i move between discrete values and features swap places. Each jump comes with a sudden drop in the loss: a very small one at the first jump and a larger one at the second. The authors suspect that seemingly smooth loss curves of larger models are made of many such small jumps. They compare this with other sudden changes during training: the formation of induction heads (Olsson et al.), phase changes in modular arithmetic (Nanda and Lieberum), and grokking (Power et al.).

Learning as geometric transformations (pp. 27–28). With $m = 3$ the arrangement of the feature directions can be watched directly. The authors train $n = 6$ features in two correlated sets of three; in a correlated set, all features are zero together or active together, with

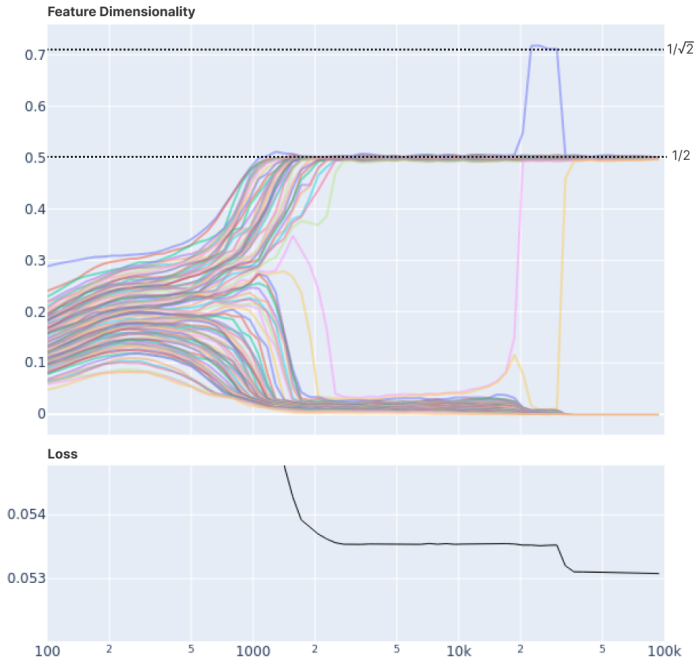


Figure 1: The paper's figure (p. 27), taken from its PDF. Top: each coloured line is the dimensionality D_i of one feature during training (steps on a log scale); the dotted lines mark $1/2$ and $1/\sqrt{2}$. Features end at $1/2$ (antipodal pairs) or 0. Near step $2 \cdot 10^4$ one feature (pink) jumps from near 0 to $1/2$ while another (light blue) rises from $1/2$ to $1/\sqrt{2}$. Near step $3 \cdot 10^4$ the light blue feature drops back to $1/2$ as a third (orange) jumps from near 0 to $1/2$. Bottom: the loss, which drops very slightly at the first jump and more at the second.

independent values (p. 23). The run shown in the paper passes through four stages, each a separate regime of the loss curve (Figure 2):

A. the weights start small and random;

B. the two sets push apart along one axis;

C. each set expands into a triangle;

D. the triangles rotate into an antiprism.

The antiprism is an octahedron in which each antipodal pair joins features from different sets. This end state attracts strongly, but runs can take other routes, which differ at stage C: sometimes the model goes straight into the antiprism, or first forms antipodal pairs. The authors guess that this depends on which geometry the model is closest to when stage B ends (p. 28).

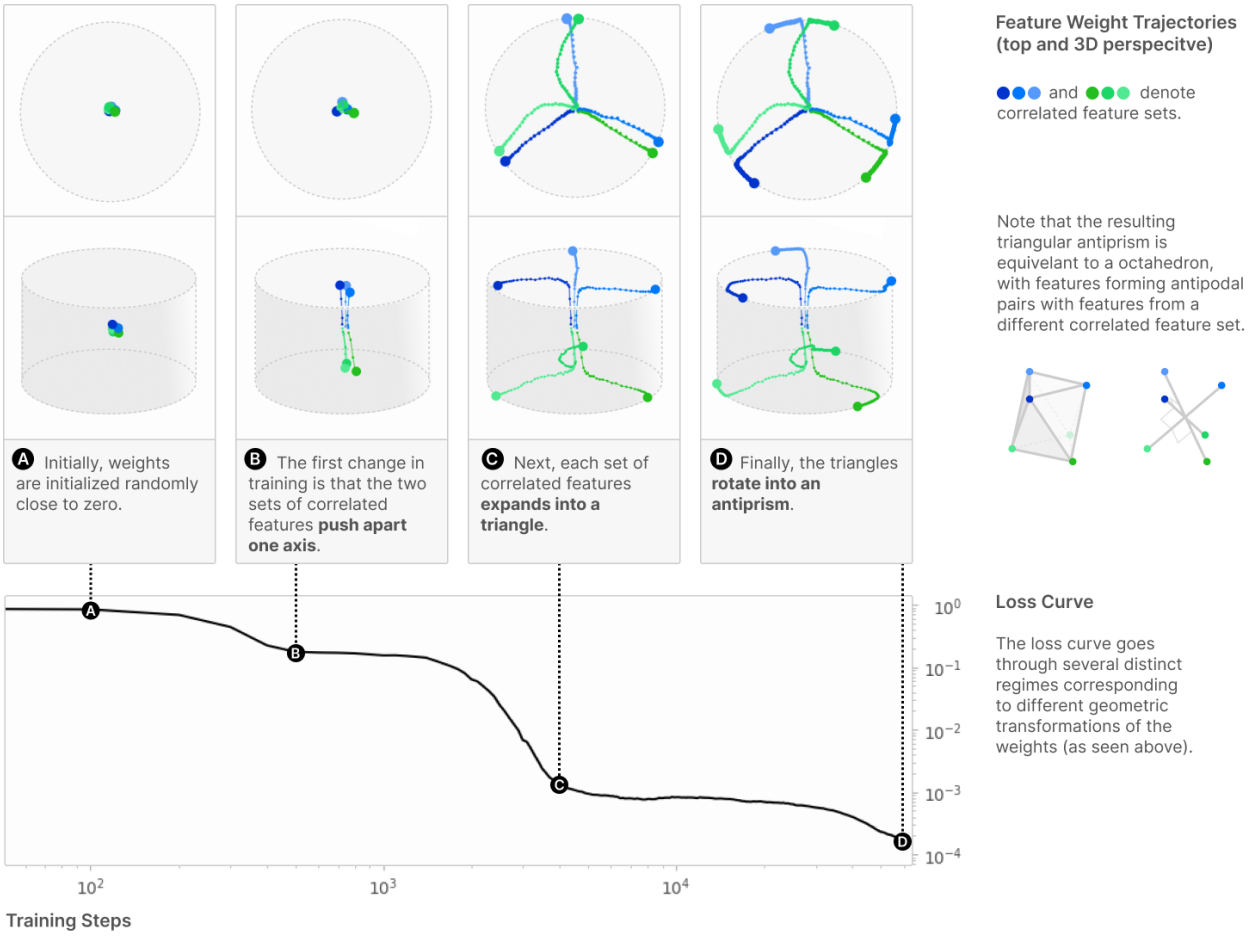


Figure 2: The paper's figure (p. 28), taken from its PDF: one training run with $n = 6$ features in $m = 3$ dimensions, in two correlated sets (blue, green). Upper panels: the paths of the feature directions during training, seen from above and in perspective. A–D mark the stages, also marked on the loss curve. Right: the final triangular antiprism is an octahedron.

3 *Repeated Experiments*

I repeated both experiments of Section 2. The paper's text does not give its optimizer, learning rate, batch size or initialization, or the sparsity used for the figures in Section 2, so those settings are my choices.

3.1 Setup common to both training experiments

- Model: the paper's ReLU output model, $x' = \mathrm{ReLU}(W^\top W x + b)$. Loss: the squared error, summed over features and averaged over the batch, with all $I_i = 1$.
- Data: a fresh batch of 1024 inputs at every step, drawn from the paper's distribution.
- Training: Adam with learning rate 10^{-3} ($\beta_1 = 0.9$, $\beta_2 = 0.999$), with exact gradients written out by hand in NumPy.
- Start: entries of W drawn from a normal distribution with mean 0 and standard deviation $\sqrt{2/(n+m)}$ times a scale factor, and $b = 0$.
- Measurements: at 120 to 150 steps spaced evenly on a log scale, the loss on a fixed set of 20,000 inputs, every D_i, and W itself.

3.2 Energy-level jumps

SETUP. $n = 80$ features in $m = 6$ dimensions; every feature is active with probability $1/3$ (so $1 - S = 1/3$); scale factor 1; 50,000 steps; four runs with different random seeds. This is a scaled-down version of the paper's uniform setting ($n = 400$, $m = 30$, p. 17), with the same ratio $n/m \approx 13$. At this sparsity every feature ends either in an antipodal pair or unrepresented, as in the model the paper picked.

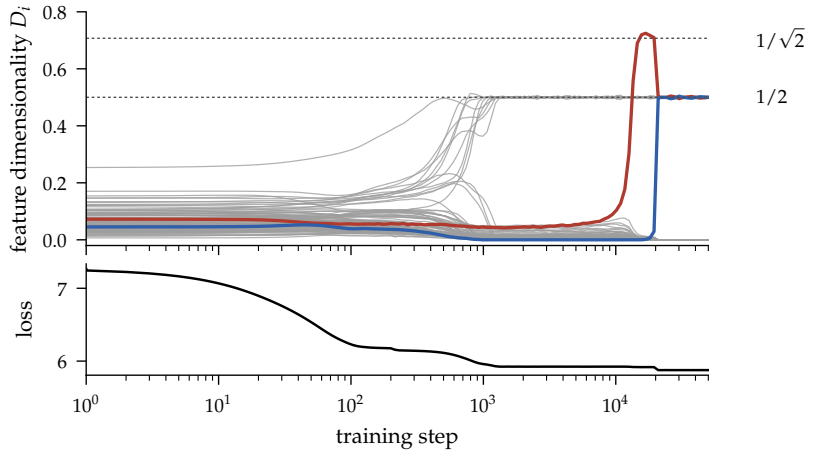


Figure 3: My experiment (seed 1 of 4). Top: the dimensionality D_i of each of the 80 features during training. Red and blue: the two features that form the sixth antipodal pair late in training. The dotted lines mark $1/2$ and $1/\sqrt{2}$, as in Figure 1. Bottom: the loss on a fixed set of inputs.

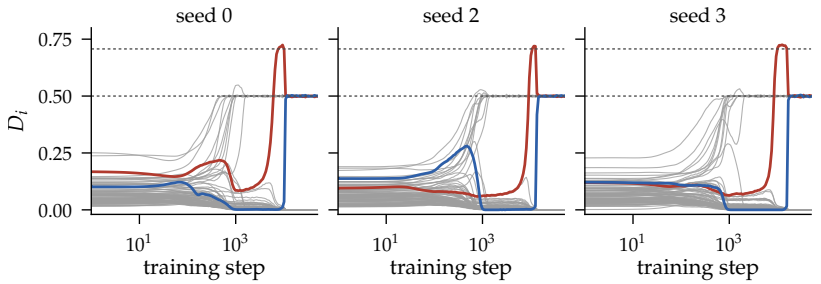


Figure 4: My other three runs (seeds 0, 2 and 3), drawn like the top of Figure 3. The swap happens near steps 10,900, 13,500 and 16,800.

RESULT (Figure 3, seed 1).

1. By step 2000, ten features sit at $D_i = 1/2$. They form five antipodal pairs, which fill five of the six dimensions. The other 70 features are near 0, and 68 of them point weakly in the same direction along the sixth dimension.

2. From about step 10^4, one feature (red) grows and takes over the sixth dimension. Its D_i rises to about 0.71, not 1, because about 50 weak features still point the same way with small weights.

3. Near step $2{\cdot}10^4$, another feature (blue) grows in exactly the opposite direction. When it reaches full length, red and blue form the sixth antipodal pair: red drops to $1/2$, blue jumps from 0 to $1/2$, and the weak features in that direction vanish.

4. The loss drops at that step, from 5.917 to 5.876, because 12 features are now represented instead of 11.

All four runs show the same sequence, at different steps (Figure 4).

3.3 Geometric transformations

SETUP. $n = 6$ features in $m = 3$ dimensions. Features 1–3 form one correlated set and features 4–6 the other, defined as in the paper (p. 23); each set is active with probability $1 - S$, independently of the other. I used $1 - S = 0.05$ and 0.1, with two seeds each; scale factor 0.1, so that the weights start small, as in the paper's stage A; 30,000 steps.

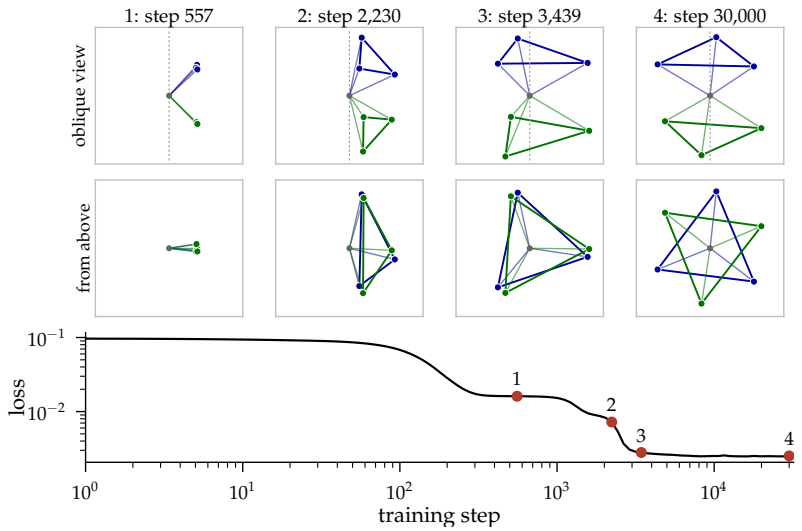


Figure 5: My experiment: the setup of Figure 2 with $1 - S = 0.05$ (seed 0). Blue: features 1–3 (one correlated set); green: features 4–6. Each point is a feature direction W_i, joined to the origin and to the other features of its set. All panels use one fixed frame; the dashed axis is the direction that separates the two sets at the end. 1: each set merged into one direction, the two at 90°. 2: the sets spread out and move apart. 3: two aligned triangles on opposite sides. 4: the triangles have rotated into the antiprism, an octahedron. Bottom: the loss, with the four snapshots marked.

RESULT (Figure 5). All four runs ended in the paper's octahedron, and all four took the same route:

1. In the first 400 steps, the three features of each set merge into one direction, and the two directions end up at 90° to each other. This is close to the PCA solution, and the loss stays flat there until about step 1000.

2. From about step 1000 to 4000, each set spreads into a triangle while the two sets move to opposite sides. The two triangles are nearly aligned.

3. From about step 4000 to 14,000, the triangles rotate against each other into the antiprism. This lowers the loss by less than 10%.

COMPARISON WITH THE PAPER. My stages 2 and 3 match the paper's stages C and D. Stage 1 matches only in part. In both, the features of each set first share one direction; the paper describes this early phase as a linear PCA solution (p. 28). But in the paper's stage B the two directions are opposite, on one axis, while in my runs they are at 90°

and become opposite only while the triangles form. Also, the paper's loss falls by a factor of about 5 between C and D, while my stage 3 lowers it by less than 10%. I have not tested whether a different sparsity or starting scale would reproduce the paper's stage B.

4 *Superposition and Adversarial Examples*

The idea (p. 29). Output i of the model is

$$x'_i = \mathrm{ReLU}\big((W^\top W)_i \cdot x + b_i\big),$$

where $(W^\top W)_i$, row i of $W^\top W$, says how strongly each input feature feeds output i. Take output 0. Without superposition its row is $(1, 0, 0, \dots)$: it reads only feature 0. With superposition the row looks like $(1, \epsilon, -\epsilon, \epsilon, \dots)$: output 0 also picks up a little of many other features (Figure 6).

Each small entry gives an attacker a way to move output 0 without touching feature 0. A small change to one other feature does little, but small changes to many of them add up. The authors point out that more training data might not remove this weakness. Superposition is the model's best response to sparse features, so a model fit to unlimited sparse data would still use it.

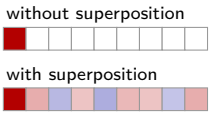


Figure 6: Row 0 of $W^\top W$: the weights with which output 0 reads the inputs (schematic, after p. 29). The first entry is 1. Without superposition every other entry is 0. With superposition many small entries appear; pale red is positive, pale blue negative.

The experiment (pp. 29–30). The authors attack trained toy models and measure how much damage the attack does. The attacker may add any vector δ to the input, as long as its length $\|\delta\|_2$ is at most 10% of the average length of an input.

The usual way to find such an attack is to follow the gradient of the loss, but here it worked poorly. The inputs are so sparse that the output ReLUs are switched off about 99% of the time. A switched-off ReLU has zero gradient, so the gradient gives no direction to push in. This problem is known as gradient masking.

So the authors use a formula instead. Among all changes of length λ, the one that moves the input to output i's ReLU the most is a step along row i of $W^\top W$:

$$\delta = \lambda \frac{(W^\top W)_i}{\|(W^\top W)_i\|_2}.$$

They compute this attack for every feature i and keep the one that raises the loss the most. They then report a damage ratio: the increase in loss under the attack, $L' - L$, divided by the same increase for a model without superposition, $L'_0 - L_0$. A ratio of 3 means that the attack does three times as much damage as it would to a model without superposition.

The result (Figure 7). As the data get sparser and the model starts to use superposition, the damage ratio goes up. It is a little above 3 while the features sit in antipodal pairs, and it reaches about 8 at the sparsest setting shown. It rises together with the number of features

per dimension, $1/D^* = \|W\|_F^2/m$, the measure of superposition from Part I. Plotted against each other, the two lie close to a straight line (Figure 8).

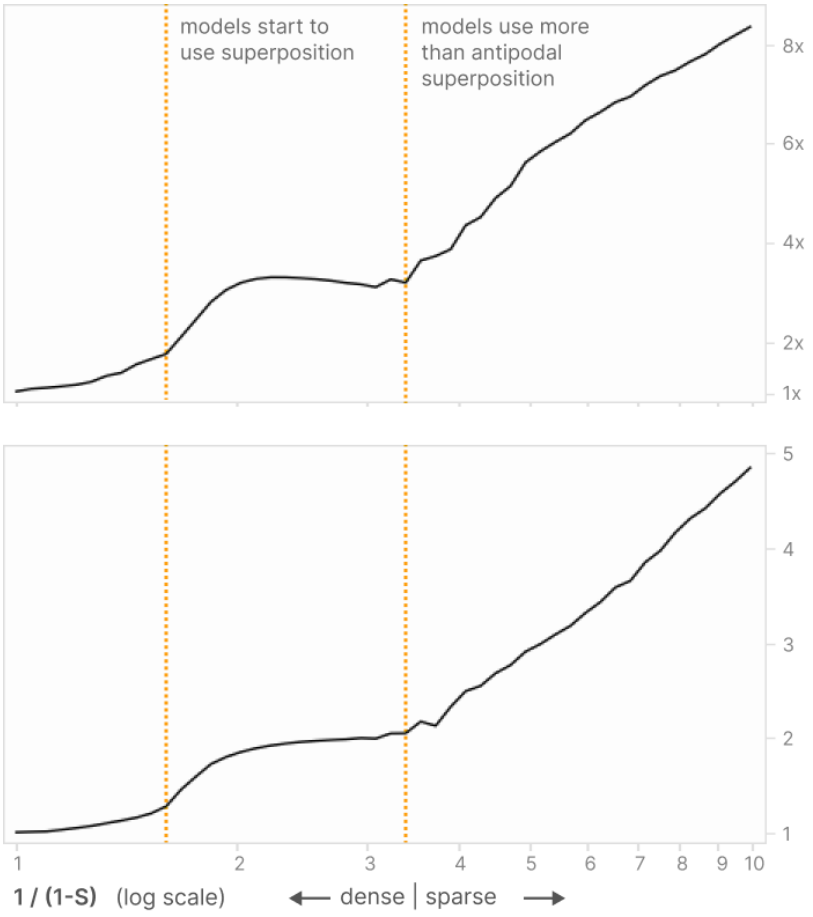


Figure 7: The paper's figure (p. 29), taken from its PDF without the paper's side notes. The horizontal axis is $1/(1-S)$, so the data get sparser to the right. Top: the damage ratio $(L'-L)/(L'_0-L_0)$. Bottom: the number of features per dimension, $1/D^* = \|W\|_F^2/m$. The dotted lines mark where the model starts to use superposition and where it starts to use more than antipodal pairs.

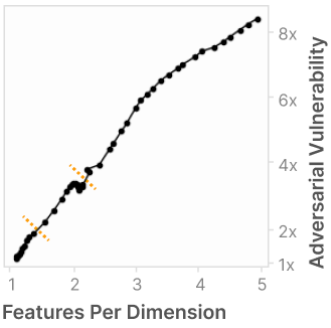


Figure 8: The paper's figure (p. 30), cropped from its PDF: the damage ratio plotted directly against the number of features per dimension.

An example to illustrate . Suppose there are only two features in one dimension (n=2, m=1). The attack moves the input to output i's ReLU by $\lambda\|(W^\top W)_i\|_2$, so the length of the row decides how strong the attack is. A small example shows where the extra strength comes from. Take an antipodal pair: features 0 and 1 share one dimension, with $W_0 = 1$ and $W_1 = -1$, so row 0 of $W^\top W$ is $(1, -1)$.

- Without superposition, the best attack on output 0 changes x_0 by λ, and this moves output 0 by λ.
- With the pair, the attacker can change x_0 by $\lambda/\sqrt{2}$ and x_1 by $-\lambda/\sqrt{2}$. This change has the same length λ, but it moves output 0 by $\lambda/\sqrt{2} + \lambda/\sqrt{2} = \sqrt{2}\,\lambda$.

So output 0 moves $\sqrt{2}$ times as far, and its squared error grows about twice as much. In general, the length of the row is set by the feature's dimensionality:

Lemma 4.1 (my addition)**.** *If* $\|W_i\| = 1$*, then* $\|(W^\top W)_i\|_2 = 1/\sqrt{D_i}$.

Proof. The entries of row i are $(W^\top W)_{ij} = W_i \cdot W_j$, and $W_i = \hat{W}_i$ because $\|W_i\| = 1$. So $\|(W^\top W)_i\|_2^2 = \sum_j (\hat{W}_i \cdot W_j)^2 = \|W_i\|^2 / D_i = 1/D_i$. □

So if output i is active, the attack raises its squared error by about λ^2/D_i instead of λ^2. The factor $1/D_i$ is the number of features per dimension for feature i: 2 for an antipodal pair, 5/2 for a pentagon and 8/3 for a square antiprism. This estimate ignores the bias, the ReLU and the effect of the attack on the other outputs. It accounts for the rise of the damage with the number of features per dimension, but not for the paper's numbers, which are larger: about 3 at two features per dimension, where the estimate gives 2.

How far the authors take this (p. 30). The authors do not claim that superposition is the main cause of adversarial examples in real networks, since there are good explanations that do not involve superposition. But they note that a "superposition maximalist" view would predict several known facts about adversarial examples:

- that they exist at all;
- that models made robust to them perform worse, since robustness means giving up superposition and representing fewer features;
- that robust models might be more interpretable;
- arguably, that they transfer between models, if which features share dimensions depends mostly on which features are correlated; models trained on the same data would then arrange their features in similar ways.

The authors also tried the reverse direction: training the model against the attack above, aimed at a random feature each time. This did reduce superposition, but removing it completely needed attacks as large as 80% of the input's length, which the authors did not find satisfying. They suggest that stronger attacks might work better, but did not pursue this.

5 Superposition in a Privileged Basis

Adding neurons (p. 31). In the ReLU output model, $W \mapsto OW$ with O orthogonal gives the same model, so the hidden space has no privileged basis, like word embeddings or the residual stream. To model layers with neurons, the paper gets a privileged basis by putting a ReLU on the hidden layer (an L1 penalty on h would be another way):

$$h = \mathrm{ReLU}(Wx), \qquad x' = \mathrm{ReLU}(W^\top h + b).$$

Now W maps features to neurons and can be read directly. The features align with neurons in a structured way, and many neurons are dedicated to a single feature. This is what justifies reading networks neuron by neuron where that works.

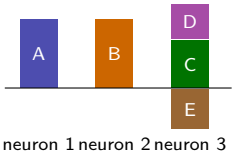


Figure 9: How to read the paper's stack plots (p. 32), with made-up weights: one bar per neuron, one rectangle per non-zero weight. Height: size of the weight; colour: the feature; negative weights below the axis; the order within a bar means nothing. Neurons 1 and 2 are monosemantic. Neuron 3 responds to C and D positively and to E negatively: polysemantic.

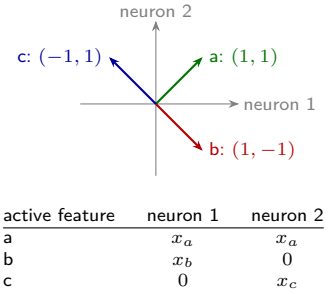
Stack plots (p. 32). For each neuron, its weights are drawn as a bar of stacked rectangles (Figure 9). In the results figures the colour shows whether a feature is in superposition rather than which feature it is, and the background marks each neuron as monosemantic or polysemantic.

Results (pp. 32–34). With $n = 10$, $m = 5$ and $I_i = 0.75^i$, the authors train 1000 models per sparsity level and show the one with the lowest loss:

$1 - S$	What the five neurons do
$\gtrsim 0.5$	5 features, each with its own neuron
0.2–0.5	3 monosemantic; 3 more features in a binary code on 2
0.15	2 monosemantic; 5 more features in 3 neurons
0.12	1 monosemantic; 6 more features in 4 neurons
0.05–0.08	8 features in 5 neurons, all polysemantic
≤ 0.04	all 10 features, each on its own pair of neurons

Neurons go from monosemantic to polysemantic as the data get sparser: at first a polysemantic neuron carries a few features, later more. Both kinds of neuron can occur in the same model, and the change looks like a phase change at the level of neurons.

The polysemantic neurons are also organized. In the second row of the table, three features share two neurons as a binary code (Figure 10). Feature a turns on both neurons, b only the first, and c only the second, so the set of neurons that are on identifies the active feature. In general a feature corresponds to a set of neurons, and a monosemantic neuron is the special case of a set with one neuron. In the last row of the table, each of the 10 features has its own pair of neurons, and 5 neurons have exactly 10 pairs. With more features, the authors expect denser binary codes.



active feature	neuron 1	neuron 2
a	x_a	x_a
b	x_b	0
c	0	x_c

Figure 10: The paper's result (p. 33), redrawn: three features a, b, c on two neurons. Top: each arrow shows one feature's weights on the two neurons. Feature a has weight $+1$ on both; b has $+1$ on neuron 1 and -1 on neuron 2; c has the reverse. So a is at $90°$ to b and c, and b and c point in opposite directions. Bottom: the outputs of the two neurons when only one feature is active, with value x_a, x_b or x_c. The hidden ReLU turns negative values into 0, so a turns on both neurons, b only neuron 1, and c only neuron 2. Which neurons are on tells which feature is active, like a two-bit binary code.

Limitation (p. 34). The hidden ReLU only constrains this model, so it avoids the ReLU when it can. Given hidden biases, it makes them positive so the neurons act linearly; without biases but with enough features, it simulates a bias by averaging many features. The next model needs its ReLU.

6 *Computation in Superposition*

Setup (pp. 34–35). The input and output are layers of a hypothetical disentangled model, and the hidden layer is the small layer we observe. The task is $y = |x|$, with active x_i uniform on $[-1, 1]$, and the weights are untied:

$$h = \mathrm{ReLU}(W_1 x), \qquad y' = \mathrm{ReLU}(W_2 h + b),$$

with importance-weighted squared error.

Since the network is monotone, one hidden neuron cannot compute $|x|$. But two neurons suffice, since $|x| = \mathrm{ReLU}(x) + \mathrm{ReLU}(-x)$ (Fig-

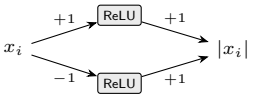


Figure 11: The paper's construction (p. 35): without superposition, $|x_i|$ takes two neurons, $|x_i| = \mathrm{ReLU}(x_i) + \mathrm{ReLU}(-x_i)$.

ure 11). With $n = 3$ and $m = 6$ the model learns exactly this, up to rescaling a hidden unit's input weights by α and its output weights by α^{-1} (fn. 15, p. 56).

Results (pp. 36–38). With $n = 100$, $m = 40$ (enough for 20 features without superposition) and $I_i = 0.8^i$:

- On dense data each neuron serves one feature.
- On sparse data the model represents more features than it has neurons while still usefully approximating $|x|$. So computation, not only storage, can happen in superposition. (Losses at different sparsities are not comparable, since the task itself changes; fn. 17, p. 56.)
- In between, many monosemantic neurons sit next to a few highly polysemantic ones, and the neurons for the most important features tend to be monosemantic. Many polysemantic neurons have one *primary* feature with a large weight and *secondary* features with small weights. Their strongest activations then come from the primary feature, as researchers have reported for neurons in language models.

Two features, without and with superposition (pp. 35, 39). The trained model's circuit is easier to read next to two simpler ones, drawn for two features x_1 and x_2 in Figure 12.

- *Without superposition* (Figure 12a). Each feature has two neurons. The neuron for its positive side has input weight $+1$ and outputs $\mathrm{ReLU}(x_i)$; the neuron for its negative side has input weight -1 and outputs $\mathrm{ReLU}(-x_i)$. Both send weight $+1$ to the output y_i, so $y_i = \mathrm{ReLU}(x_i) + \mathrm{ReLU}(-x_i) = |x_i|$. Each output reads only its own two neurons, so there is no interference, but two features use four neurons.
- *Ordinary superposition* (Figure 12b). One neuron N does two of these jobs: the positive side of x_1 and the negative side of x_2. Its input weights are $+1$ from x_1 and -1 from x_2, and its output weights are $+1$ to y_1 and $+1$ to y_2. These are the weights $[1, -1]$ and $[1, 1]$ of the paper's example. The numbers are the same as in (a), but their roles differ: in (a), $+1$ and -1 go from one feature to two neurons; here they go from two features to one neuron. This saves one neuron. Because the features are sparse, N usually serves only one feature at a time, but y_1 and y_2 cannot tell which one. If only x_1 is active and positive, N outputs x_1, which goes to y_1, as it should, and also to y_2, which should be 0. In the same way, a negative x_2 on its own also reaches y_1.

Asymmetric superposition and inhibition (pp. 38–39). The trained model that the paper examines handles this interference differently. It contains two instances of a two-neuron motif. As in Figure 12b, one neuron N handles the positive side of feature 1 and the negative side of feature 2 (Figure 13). But instead of the ordinary

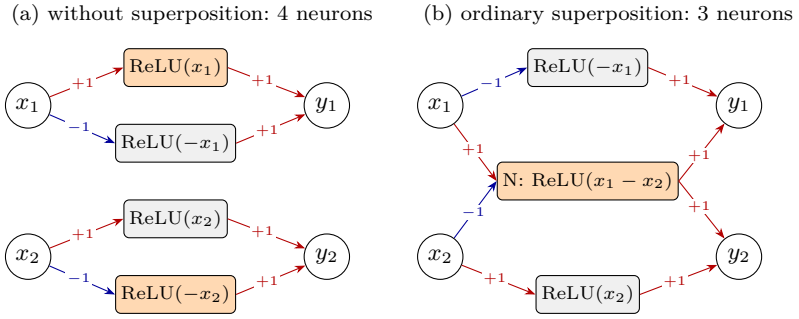
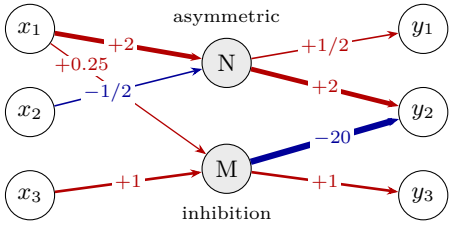


Figure 12: Computing $|x_1|$ and $|x_2|$. Red arrows: weight $+1$; blue arrows: weight -1. (a) Without superposition, as the paper describes it (p. 35): each feature has one neuron for its positive side and one for its negative side, and each output adds its two neurons. (b) Ordinary superposition: neuron N has the weights of the paper's example (p. 39), and I added the two neurons for the remaining sides. N replaces the two shaded neurons of (a). The two features now need one neuron fewer, but a positive x_1 also reaches y_2, and a negative x_2 also reaches y_1. Figure 13 shows how the paper's trained model reduces this interference.

weights $[1, -1]$ and $[1, 1]$, it uses input weights $[2, -\frac{1}{2}]$ and output weights $[\frac{1}{2}, 2]$: asymmetric superposition. If only feature i is active, on the side that N responds to, N's activation is $|w_i|\,|x_i|$, where w_i is its input weight. So the path from input i to output j has strength $|w_i|\times$ (output weight to j):

- Own paths: $2 \cdot \frac{1}{2} = 1$ and $\frac{1}{2} \cdot 2 = 1$, so each feature comes out at the right size.
- Cross paths: $x_1 \to y_2$ has strength $2 \cdot 2 = 4$, and $x_2 \to y_1$ has strength $\frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$.

Since both own paths equal 1, the two cross paths always multiply to 1: rescaling moves interference from one direction to the other but cannot remove it. Feature 1 now interferes strongly with feature 2, and feature 2 hardly at all with feature 1. A second neuron M, whose main job is another feature x_3, takes a small input from x_1 and sends a large negative weight to y_2. When $x_1 > 0$, M outweighs N at y_2, so the input to y_2's ReLU is negative and the ReLU removes the interference. In the paper's terms, costly positive interference is converted into negative interference, whose consequences the output ReLU limits. The price is that x_3 now inhibits y_2 too, which matters only when x_2 and x_3 are active together. The authors take a few remaining weights to be small conditional biases.



path strengths through N

	y_1	y_2
x_1	1	1
x_2	1	1

symmetric

	y_1	y_2
x_1	1	**4**
x_2	1/4	1

asymmetric

Figure 13: The paper's motif (pp. 38–39), redrawn: asymmetric superposition (neuron N) with inhibition (neuron M). Red: positive weights; blue: negative. N's weights are the paper's example; M's weights are made up, since the paper gives none. With $x_1 = 0.8$ alone, the symmetric neuron puts 0.8 into y_2; here N puts in 3.2 and M takes out 4, so with output bias 0 the input to y_2's ReLU is negative and $y_2 = 0$. With $x_2 = -0.8$ alone, the symmetric neuron puts 0.8 into y_1; here only 0.2.

7 *The Strategic Picture*

Why superposition matters for safety (pp. 40–41). To show that a model never does something like deliberate deception, the authors look to enumerating all its features, a universal quantifier over the

basic units of computation. The target is a claim such as *all deceptive-behavior features stay out of circuits X, Y, Z*, not that no such feature exists, since models that represent the world are expected to represent behaviors like deception (fn. 18, p. 56); enumeration would also help with unknown unknowns. With a privileged basis and no superposition, features are neurons and enumeration is easy. Conversely, knowing all feature directions would let one unfold the activations, by compressed sensing and with high probability, into those of a larger model without superposition; any method that achieves this the authors call a solution to superposition. It is also needed to decompose activation space into understandable parts, to describe activations in terms of pure features, and for circuit analysis. Even basic tools suffer: unrelated features get positive dot products, so cosine similarity can mislead, and a known feature direction cannot simply be projected out (fn. 19, pp. 56–57).

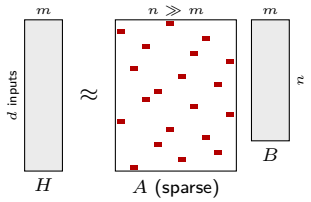


Figure 14: Schematic of the paper's footnote 20 (p. 57), the second way out as a factorization: the activations H (one row per input) are written as sparse codes A times a dictionary B whose n rows are candidate feature directions.

Three ways out (pp. 41–43). All seem feasible for toy models, hard for large ones.

1. **Models without superposition.** An L1 penalty $\lambda\|h\|_1$ on the hidden activations removes superposition from the toy models, at the cost of higher loss. Superposition probably helps real models a lot, because many features are very sparse; language models know people of modest prominence, such as several of the authors, who presumably appear far less than once per million tokens (p. 42). But the real constraint is compute, not neurons. A mixture-of-experts model activates only $1/k$ of its neurons per input, in blocks, so it gets about k times more neurons for similar compute. Superposition exploits the gap between how sparse neurons are and how sparse features are; mixture-of-experts models use the same gap, so they should show less superposition.

2. **An overcomplete basis found after training**: a sparse coding problem, $H \approx AB$ with A sparse (Figure 14). Difficulties: the number of features is unknown; the features become "virtual neurons" apart from the network's own units; factoring a millions-by-billions matrix a thousand times overcomplete is a new engineering problem; and training no longer pushes against superposition.[1]

3. **Hybrids**: reduce superposition during training and then decode, or change the architecture so that the overcomplete basis is easier to find. Some reduction, e.g. by mild L1 regularization, should be almost free. At the trained optimum s^*, the loss has zero derivative with respect to the amount of superposition s (fn. 21, p. 57), so by Taylor expansion

$$L(s^* - \varepsilon) - L(s^*) \approx \tfrac{1}{2} L''(s^*)\, \varepsilon^2,$$

   which is quadratic in the reduction ε.

[1] Outside the paper: this route was later taken with sparse autoencoders, e.g. Bricken et al., "Towards Monosemanticity" (*Transformer Circuits Thread*, 2023) and Cunningham et al., "Sparse Autoencoders Find Highly Interpretable Features in Language Models" (2023).

Further considerations (p. 44). Phase changes give hope: there is a regime with no superposition at all, if models can be pushed into

it. Even a superposition-free model with poor performance would be a research tool, a ground truth for studying superposition in other models. The local almost-orthogonal bases of correlated features (Part I) are not enough, because safety claims must hold on the whole input distribution and off it.

8 Discussion and Related Work

Evidence from real networks (p. 45). Polysemantic neurons exist, often in the same layer as monosemantic ones. InceptionV1 has more polysemantic neurons in later layers, plausibly because higher-level features are sparser. First-layer transformer MLP neurons are very polysemantic, as expected if they separate the readings of a token ("die" in English, German, Dutch, Afrikaans), which are very sparse features. The authors expect superposition, the monosemantic/polysemantic split and perhaps the adversarial link to generalize, and are much less sure about the geometry and the learning dynamics.

Selected open questions (pp. 45–46). Is there a statistical test for superposition? Can we control it (L1 penalties, adversarial training, activation functions)? Closed-form solutions beyond the partial results for $n = 2$, $m = 1$? Can the importance and sparsity curves of real features be estimated? Does superposition shrink with scale? How many features fit in superposition? Which computations can be done in superposition? Do models use nonlinear representations?

9 Questions for Discussion

- Here we are assuming the each dimension of the input represents a single feature, and the aim is to reconstruct the input. This makes polysemanticity a problematic case. In real LLMs, however, interactions of multiple features across layers are necessary. But then, how do we even pick out the (vicious) polysemanticity/superposition that we want to get rid of?